



LeetCode 200 — Number of Islands

1. Problem Title & Link

Title: LeetCode 200: Number of Islands

Link: <https://leetcode.com/problems/number-of-islands/>

2. Problem Statement (Short Summary)

You're given a grid of '1' (land) and '0' (water).

An **island** is a group of connected '1's (connected only up, down, left, right).

Your job: **Count how many separate islands exist.**

3. Examples (Input → Output)

Example 1

Input:

11000

11000

00100

00011

Output: 3

Example 2

Input:

1

Output: 1

Example 3

Input:

10

01

Output: 2

4. Constraints

- Grid size: $1 \leq m, n \leq 300$
- Only '0' and '1'
- Must visit all land
- Must avoid re-counting the same island
- DFS/BFS required
- Space: recursion/queue allowed

5. Core Concept (Pattern / Topic)



Graph Traversal — DFS / BFS

Each '1' is a node.

Island = a connected component.

Count components.

6. Thought Process (Step-by-Step)

✗ Brute Force

Count every '1' blindly → fails because

- one island might be counted many times
- doesn't track visited cells

✓ Optimized Logic (DFS / BFS)

Idea:

- Scan the grid cell by cell
- When you find a '1':
→ This is a **new island**
→ Perform DFS/BFS to mark all connected '1's as visited
- Continue scanning
- Increase island count only when new DFS/BFS starts

Why this works:

Because once you explore and mark an island, you won't count it again.

7. Visual / Intuition Diagram

Grid:

11000

11000

00100

00011

Visual grouping:

[1 1] 0 0 0

[1 1] 0 0 0

0 0 [1] 0 0

0 0 0 [1 1]

We see:

- Big island at top-left
- Single-cell island in middle
- Two-cell island bottom-right



Total → 3

8. Pseudocode

```
count = 0

for each cell (i, j):
    if grid[i][j] == '1':
        count += 1
        dfs(i, j)

function dfs(i, j):
    if out of bounds or grid[i][j] != '1':
        return

    mark grid[i][j] = '0' # visited

    dfs(i+1, j)
    dfs(i-1, j)
    dfs(i, j+1)
    dfs(i, j-1)
```

9. Code Implementation



Python

```
class Solution:
    def numIslands(self, grid):
        if not grid:
            return 0

        rows, cols = len(grid), len(grid[0])
        count = 0

        def dfs(r, c):
            if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] != '1':
                return
            grid[r][c] = '0' # mark visited
            dfs(r+1, c)
            dfs(r-1, c)
            dfs(r, c+1)
            dfs(r, c-1)

        for r in range(rows):
```

```
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                dfs(r, c)

    return count
```

✓ Java

```
class Solution {

    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;

        int rows = grid.length;
        int cols = grid[0].length;
        int count = 0;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {

                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c);
                }
            }
        }

        return count;
    }

    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] != '1')
            return;

        grid[r][c] = '0'; // mark visited

        dfs(grid, r+1, c);
        dfs(grid, r-1, c);
        dfs(grid, r, c+1);
        dfs(grid, r, c-1);
    }
}
```



}

10. Time & Space Complexity

Time: $O(m \times n)$

Every cell is visited once.

Space:

- DFS recursion: $O(m \times n)$ worst case (all land)
- BFS version would use queue: $O(\min(m,n))$ per expansion

11. Common Mistakes / Edge Cases

✗ Mistakes

- Forgetting to mark visited
- counting same island multiple times
- using diagonal connections (not allowed)
- modifying original grid incorrectly

✓ Edge Cases

- All water
- All land
- One row / one column
- Single cell

12. Detailed Dry Run (Step-by-Step)

Grid:

11000

11000

00100

00011

Step	Cell	Action	Islands	Notes
1	(0,0)=1	New island	1	DFS marks 4 cells
2	next 1s	already visited	1	skip
3	(2,2)=1	New island	2	mark it visited
4	(3,3)=1	New island	3	marks (3,3),(3,4)
End	—	Total	3	

13. Common Use Cases (Real-Life / Interview)



- Map/terrain segmentation
- Flood-fill algorithm
- Counting connected components in any graph
- Clustering pixels (image processing)
- Region detection (game development)

14. Common Traps

- Doing DFS without bounds check
- Infinite recursion due to not marking visited
- Using diagonal neighbors
- Forgetting to increment count only at start of DFS

15. Builds To (Related LeetCode Problems)

- LC 130: Surrounded Regions
- LC 417: Pacific Atlantic Water Flow
- LC 49: Number of Provinces
- LC 695: Max Area of Island
- LC 547: Friend Circles

16. Alternate Approaches + Comparison

1 BFS Instead of DFS

Same logic, uses a queue.

2 Union Find (Disjoint Set)

- Treat each land cell as a node
- Union neighboring lands
- Count number of roots
- More complex but works well for huge grids

DFS is simplest and fastest for interviews.

17. Why This Solution Works (Short Intuition)

DFS wipes out (marks visited) an entire island in one go.

So every time we encounter a '1', it's guaranteed to be a **new island**.

18. Variations / Follow-Up Questions

- Count islands including diagonal connections
- Return largest island area



- Convert water to land dynamically (LC 305)
- Replace recursion with iterative DFS